

Lernziele Exceptions

- **Preconditions prüfen (Fail Fast):**
 - Parameter auf Gültigkeit testen (z. B. `null-Prüfung`, `isEmpty()`).
- **Exceptions werfen:**
 - **Unchecked:** `throw new IllegalArgumentException()` bei ungültigen Parametern.
 - **Checked:** `throw new EigeneException()` bei erwarteten Fehlern.
- **Eigene Checked Exceptions:**
 - Klasse erstellen, die von `Exception` erbt.
 - In Methodensignatur mit `throws EigeneException` deklarieren.
- **Exceptions behandeln:**
 - Risikocode mit `try-catch` absichern.
 - **Umwandlung (Wrapping):** `catch (SomeException e) und throw new MyException(e).`

50

Punkt 1: Preconditions prüfen (Fail Fast)

Der erste wichtige Punkt ist das Prüfen von 'Preconditions', also Vorbedingungen. Das knüpft direkt an das 'Fail Fast'-Prinzip an, das wir auf Folie 19/20 besprochen haben.

Was bedeutet das? Ganz einfach: Bevor eine Methode ihre eigentliche Arbeit beginnt, sollte sie *zuerst* prüfen, ob sie überhaupt gültige Daten erhalten hat. Ist ein Parameter null, obwohl er nicht null sein darf? (Wie in der Aufgabe 'Concat Strings')

Ist eine Collection `isEmpty()`, obwohl sie Daten enthalten muss? (Wie in der Aufgabe 'Average Exception')

Wenn wir das *sofort* am Anfang prüfen und die Methode mit einer Exception abbrechen, verhindern wir zwei Dinge:

Dass die Methode mit ungültigen Daten weiterarbeitet und erst viel später an einer ganz anderen Stelle einen kryptischen Fehler wirft.

Dass unser System in einen inkonsistenten Zustand gerät.

Fehler so früh wie möglich zu erkennen, macht das Debugging massiv einfacher. Das ist defensives Programmieren."

Punkt 2: Exceptions werfen (Unchecked vs. Checked)

"Wenn eine solche Vorbedingung fehlschlägt, müssen wir eine Exception werfen.

Hier unterscheiden wir zwei Arten:

Unchecked Exceptions, wie `IllegalArgumentException`: Diese verwenden wir, wenn der *Aufrufer* der Methode einen Fehler gemacht hat – einen Programmierfehler. Er hat uns ungültige Parameter gegeben. Diese Exceptions erben von `RuntimeException` und der Aufrufer *muss* sie nicht mit try-catch behandeln.

Checked Exceptions, wie unsere `EigeneException`: Diese verwenden wir für Fehler, die *außerhalb* unserer Programmlogik liegen, aber erwartet werden können. Zum Beispiel: Eine Datei wird nicht gefunden, das Netzwerk ist offline, die Datenbank antwortet nicht. Das sind keine Programmierfehler, sondern externe Umstände, auf die der Aufrufer sinnvoll reagieren *muss*. Deshalb *zwingt* uns der Compiler hier zur Behandlung."

Punkt 3: Eigene Checked Exceptions

"Um solche erwarteten Fehler klar zu benennen, definieren wir eigene Checked Exceptions. Das haben wir in der Aufgabe 'Double Converter' mit der `IsNotANumberException` geübt.

Das ist ganz einfach:

Wir erstellen eine neue Klasse, die von `Exception` erbt (oder einer spezifischeren Subklasse wie `IOException`).

Wichtig: Jede Methode, die diese Exception *werfen könnte*, muss dies in ihrer Signatur mit dem throws-Schlüsselwort deklarieren.

Dadurch machen wir die Fehlermöglichkeit explizit und zwingen den Aufrufer (wie auf Folie 3 gezeigt), sich damit auseinanderzusetzen."

Punkt 4: Exceptions behandeln (try-catch & Umwandlung)

"Der letzte Punkt ist die Reaktion auf Exceptions.

Der try-catch-Block ist der Standardweg, um mit Checked Exceptions umzugehen, wie wir es auf Folie 8 in der `MainView` gesehen haben. Wir *versuchen* den riskanten Code auszuführen und *fangen* den Fehler ab, falls er auftritt, um z.B. eine Nutzermeldung anzuzeigen.

Besonders wichtig ist die **Umwandlung (Wrapping)**, die wir auf Folie 14 besprochen haben. Stellen Sie sich vor, unsere Datenschicht wirft eine technische `IOException` oder `NumberFormatException` (wie in der Aufgabe 'Double Converter'). Die Business-Schicht darüber sollte diese technischen Details nicht kennen.

Deshalb fangen (catch) wir die technische Exception und werfen (throw) stattdessen unsere eigene, semantisch passendere Exception (z. B. `DatabaseException` oder `NotANumberException`).

Damit wahren wir die Abstraktion zwischen den Schichten und verhindern, dass Implementierungsdetails 'durchsickern'."

Lernziele Optional

- **Zweck:**
 - Typsichere Alternative zu `null` *nur* für **Rückgabewerte**.
- **Optional erzeugen:**
 - `Optional.empty()`: Für "kein Ergebnis".
 - `Optional.of(value)`: Für "garantiertes Ergebnis".
- **Funktional verarbeiten (statt `isPresent()`):**
 - `.orElse(defaultValue)`: Standardwert bei `empty`.
 - `.map(lambda)`: Wert im `Optional` transformieren.
- **Streams & Optional:**
 - Stream-Methoden wie `.findFirst()` oder `.average()` geben `Optional` zurück.
- **Wann NICHT verwenden:**
 - Nie für Parameter.
 - Nie für Collections (`Optional<Collection>`). **Besser:** Leere Collection zurückgeben.

51

Punkt 1: Zweck

"Kommen wir zu `Optional<T>`. Warum gibt es das überhaupt? Wegen des 'Billion-Dollar-Mistake' (Folie 22): `null`. `null` führt zu `NullPointerExceptions` zur Laufzeit. `Optional` ist eine typsichere Alternative zu `null`. Es ist ein 'Container', der ein Objekt enthalten *kann* oder eben *nicht*. Der Compiler *zwingt* uns, den Fall zu behandeln, dass der Container leer ist.

Das Wichtigste, wie auf Folie 32 betont: `Optional` ist **primär als Rückgabebetyp** gedacht. Es drückt explizit aus: 'Diese Methode findet vielleicht nichts..'

Punkt 2: Optional erzeugen

"Wie erzeugen wir ein `Optional`? Statt `return null` verwenden wir:

`Optional.empty()`: Das ist der 'neue' Weg, um 'kein Ergebnis' zurückzugeben. (Genutzt in 'Average Optional').

Statt `return value` verwenden wir:

`Optional.of(value)`: Das 'verpackt' unser Ergebnis in den `Optional`-Container.

Wichtig: `value` darf hier *nicht* `null` sein, sonst wirft `of` selbst eine `NullPointerException`. (Genutzt in 'Average Optional').

(Es gibt auch `Optional.ofNullable(value)`, das `value` prüft und bei `null` automatisch `Optional.empty()` zurückgibt – sehr praktisch an Schnittstellen zu altem Code.)"

Punkt 3: Funktional verarbeiten (statt isPresent())

"Jetzt der wichtigste Teil: Wie nutzen wir ein Optional?"

Der 'schlechte' Stil (Folie 31) wäre: `if (opt.isPresent()) { opt.get() }`. Das ist kein bisschen besser als eine null-Prüfung.

Der 'gute', funktionale Stil nutzt Methoden, die das Optional direkt verarbeiten:

`.orElse(defaultValue)`: Das ist der einfachste Fall. 'Gib mir den Wert, und falls er leer ist, nimm stattdessen diesen Standardwert.' (Genutzt in 'Average Optional').

`.map(lambda)`: Das ist extrem mächtig. 'Wenn ein Wert vorhanden ist, wende diese Funktion darauf an und gib das Ergebnis als *neues* Optional zurück. Wenn er leer ist, bleibt er leer.' (Genutzt in 'Find Student ID', um vom `Optional<Student>` zur `Optional<Integer>` ID zu kommen)."

Punkt 4: Streams & Optional

"Optional und Streams arbeiten perfekt zusammen. Viele Stream-Operationen geben von Natur aus Optional zurück.

Warum? Weil eine Stream-Operation nicht immer ein Ergebnis garantiert.

`.findFirst()` (genutzt in 'Find Student ID') findet vielleicht kein passendes Element.

`.average()` (genutzt in 'Average Optional') kann keinen Durchschnitt einer leeren Liste berechnen.

Das Ergebnis dieser Operationen ist logischerweise ein Optional, das wir dann direkt funktional (mit `.map()` oder `.orElse()`) weiterverarbeiten können."

Punkt 5: Wann NICHT verwenden

"Zuletzt die Anti-Patterns – wann Optional *nicht* verwendet werden sollte (Folie 32 & 35).

Nie für Parameter: Optional als Methodenparameter ist kompliziert und unnötig. Nutzen Sie stattdessen Methoden-Überladung, wenn ein Parameter optional ist.

Nie für Collections: Ein `Optional<Collection>` ist fast immer falsch. Eine Methode, die eine Liste von Dingen sucht (wie in 'Finde Duplikate'), sollte *niemals* null oder `Optional.empty()` zurückgeben, wenn sie nichts findet. Sie sollte einfach eine **leere Collection** zurückgeben. Das ist viel einfacher für den Aufrufer, der dann immer über das Ergebnis iterieren kann, ohne eine null-Prüfung machen zu müssen."